Introduction to Deep Reinforcement Learning

Notes by Marco Galletti

Lecturer: Georgina Nouli

Technical University of Munich
Winter Semester 2026/27

Contents

1 Search Algorithms **2**
- 1.1 Search problems . . . 2
- 1.2 Uninformed and informed search . . . 3
- 1.3 Adversarial and expectimax search . . . 8

2 Markov Decision Processes **10**
- 2.1 MDP model and policies . . . 10
- 2.2 Utilities and the Bellman equations . . . 12
- 2.3 Value iteration and policy iteration . . . 14

1 Search Algorithms

Decision making is first studied from the optimisation side: a deterministic problem is modelled as a graph and solved by searching it for a (least-cost) path to a goal; at the end, an adversary and then randomness are added.

1.1 Search problems

Definition 1.1 (Algorithm, completeness, optimality)**.** An **algorithm** is a finite sequence of well-defined, computer-implementable instructions that solves a specific set of computable problems: it takes a finite input, processes it unambiguously at each operation and returns its output within a finite amount of time. An algorithm is **complete** if, on any input for which a solution exists, it finds one, and **optimal** if it is guaranteed to find the best feasible solution. A **heuristic** is an algorithm that is not guaranteed to be optimal.

Definition 1.2 (Graphs, paths, trees)**.** A **graph** consists of **vertices** (**nodes**), representing objects, and **edges**, representing relations between them; edges may be directed or undirected, **arcs** are always directed, and **weights** on the edges describe the relations (e.g. costs). A **path** is a sequence of nodes in which consecutive nodes are joined by an edge. A **shortest path** between two nodes is one with the fewest edges (actions), a **least-cost path** one with the least sum of weights; they coincide when all weights are 1. A **cycle** is a non-empty path whose only repeated vertices are the first and the last. A **root** is a vertex with only outgoing edges, and a **rooted graph** is a graph in which one vertex is the root. A **tree** is a cycle-free rooted graph with exactly one path from the root to each vertex.

For instance, if a start node is joined to a goal directly with weight 6 and through a node A with weights 1 and 2, the shortest path is the direct edge (one action), the least-cost path goes through A (cost 3).

Definition 1.3 (Search problem)**.** A **search problem** consists of a **state space** S with **states** $s \in S$; for each state an **action space** $A(s)$ with **actions** $a \in A(s)$; a **successor function** T, giving the state $T(s, a) \in S$ reached by taking $a \in A(s)$ in s, together with the cost of the action; costs are nonnegative; a **start state** $s_0 \in S$; and a **goal test**, which tells whether a state is a goal. A **solution** (or **plan**) is a sequence of actions $\pi = (a_1, \dots, a_k)$ that transforms the start state into a goal:

$$s_i = T(s_{i-1}, a_i), \quad a_i \in A(s_{i-1}) \quad (i = 1, \dots, k), \qquad s_k \text{ is a goal.}$$

Its cost is the sum of the costs of its actions; a least-cost solution is denoted π^*.

Example 1.4. In a graph with states $S, A, \dots, G$, start S and goal G, let the action $\mathrm{go}(X)$ move along an edge to X (so $T(S, \mathrm{go}(C)) = C$). If the graph has the edges $S \to C$ and $C \to G$, then $(\mathrm{go}(C), \mathrm{go}(G))$ is a solution. Likewise, travelling from Munich to Berlin has cities as states and trips to another city as actions, and a solution is a travel plan; in Pac-Man a state is the position of Pac-Man, the positions of the monsters and the subset of remaining food dots, the actions are North, South, East, West.

State space graphs and search trees. A search problem can be represented in two ways. In the **state space graph** the nodes are the states, the root is the start state, the arcs are the transitions to successors (actions), the goal test is a set of goal nodes (possibly one) and a solution is a path in the graph: *each state occurs only once.* In the **search tree** the root is the start state, the children of a node are its successors, the goal nodes are leaves and a solution is a path in the tree: *a state can occur several times*, once for every path that reaches it (from Munich, Berlin can be reached through Leipzig, through Nuremberg and Leipzig, through Hanover, …). For large problems neither can be built completely in an efficient way.

Definition 1.5 (Tree search, graph search)**.** A search algorithm explores a search graph or tree systematically, starting from the start state. **Expanding** ("visiting") a node means generating its successors. The nodes generated but not yet expanded form the **frontier**, and the data structure of the frontier (queue, stack, …) determines the order of exploration. **Tree search** may expand a state several times. **Graph search** expands each state at most once: it is tree search plus a **marked list** of the states already expanded.

The representation of the problem and the algorithm are independent of each other, and any combination is possible: a search tree can also be explored by graph search. For instance, if S leads to A, B and C, each of these leads to D, and then $D \to E \to G$, the search tree contains D and E three times; tree search may expand them three times, while graph search expands them once. An optimal search algorithm finds least-cost plans.

1.2 Uninformed and informed search

Uninformed search uses only the problem definition. All algorithms below are run as graph search on the graph of fig. 1, start S, goal G, with children taken in alphabetical order. Depth-first and breadth-first search ignore the weights. In the traces the frontier is listed in the order in which its nodes will be removed. The pseudocode stops at the goal; the returned path is the path to the goal in the search tree, read backwards from the goal through the node each node was generated from.

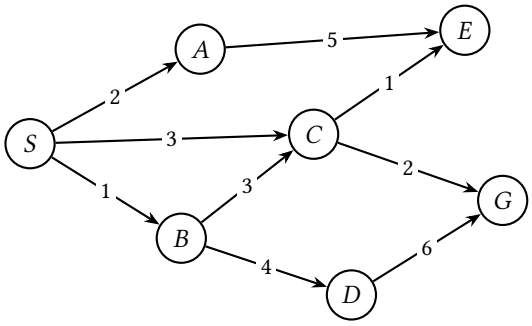


Figure 1: Example graph for uninformed search.

Depth-first search (DFS). DFS explores the children of the active node first: its strategy is to **expand a deepest node first**. Recursively:

Algorithm 1 Depth-first (graph) search, called as DFS(G, s_0)

```
1: procedure DFS(G, v)                          ▷ G: search graph, v: current node
2:     mark v
3:     if v is a goal then terminate
4:     end if
5:     for each child c of v in G do
6:         if c is not marked then DFS(G, c)
7:         end if
8:     end for
9: end procedure
```

When a node has no unmarked children and the search has not terminated, the recursion returns to the caller, i.e. to the direct predecessor in the graph. Equivalently, the frontier is a **LIFO stack**: pop the top node; if it is a goal, stop; otherwise, if it is not marked, mark it and push its unmarked children, the alphabetically first one last, so that it is on top. On fig. 1 (table 1) the stack holds C twice after expanding

B, since C was pushed both by S and by B; the recursive version marks the same states in the same order S, A, E, B, C, G. DFS returns $S \to B \to C \to G$ (three actions).

Popped	Stack after the step (top first)	Marked
S	A, B, C	S
A	E, B, C	S, A
E	B, C	S, A, E
B	C, D, C	S, A, E, B
C	G, D, C	S, A, E, B, C
G	goal: return $S \to B \to C \to G$	

Table 1: DFS with a stack on fig. 1.

Breadth-first search (BFS). BFS explores the neighbours first: its strategy is to **expand the shallowest node first**, with the frontier a **FIFO queue**. The operation *dequeue* returns and removes the first element of the queue, *enqueue* inserts an element at its end, so the node that has been in the queue longest is expanded first.

Algorithm 2 Breadth-first (graph) search

Require: search graph G, start node s_0
if s_0 is a goal **then terminate**
end if
$Q \leftarrow$ queue containing s_0
while Q is not empty **do**
    $v \leftarrow Q$.dequeue()
    if v is not marked **then**
        mark v
        for each child c of v in G **do**
            if c is a goal **then terminate**
            end if
            Q.enqueue(c)
        end for
    end if
end while

BFS tests a node for the goal when it is generated. On fig. 1 (table 2) it returns $S \to C \to G$, two actions.

Dequeued	Queue after the step (front first)	Marked
S	A, B, C	S
A	B, C, E	S, A
B	C, E, C, D	S, A, B
C	E, C, D, E; child G is a goal: return $S \to C \to G$	S, A, B, C

Table 2: BFS on fig. 1.

Uniform-cost search (UCS). BFS finds a shortest path (fewest actions), but not necessarily a least-cost path.

Example 1.6. Let $S \to A$ cost 1, $A \to B$ cost 2, $B \to G$ cost 1 and $S \to G$ cost 5. BFS generates G as a child of S and returns $S \to G$ with cost 5, while the best solution $S \to A \to B \to G$ costs $1 + 2 + 1 = 4$; UCS finds the latter.

UCS explores the cheapest nodes first: its strategy is to **expand the cheapest node first**, i.e. the one with the least cumulative path cost $g(v)$ from the start, and its frontier is a **priority queue** ordered by g. The operation $replace(c, k)$ replaces the cost stored for c in the queue by k.

Algorithm 3 Uniform-cost (graph) search

Require: search graph G with weights w, start node s_0

$Q \leftarrow$ priority queue containing s_0 with $g(s_0) = 0$
while Q is not empty **do**
  $v \leftarrow Q$.pop() ▷ node of least g
  if v is a goal **then terminate**
  end if
  mark v
  for each child c of v in G **do**
    $g' \leftarrow g(v) + w(v, c)$ ▷ cost to c through v
    if c is not marked and not in Q **then**
      Q.insert(c, g')
    else if c is in Q with $g(c) > g'$ **then**
      Q.replace(c, g')
    end if
  end for
end while

Unlike BFS, UCS tests for the goal when a node is popped, not when it is generated; this is what makes it find a least-cost path. After a replace, the predecessor of c on its current cheapest path is v. On fig. 1 (table 3, ties broken alphabetically) it returns $S \to C \to G$ with cost 5.

Popped (g)	Priority queue after the step, $c\,(g)$	Marked
$S\,(0)$	$B\,(1), A\,(2), C\,(3)$	S
$B\,(1)$	$A\,(2), C\,(3), D\,(5)$ (C through B: $4 > 3$, kept)	S, B
$A\,(2)$	$C\,(3), D\,(5), E\,(7)$	S, B, A
$C\,(3)$	$E\,(4), D\,(5), G\,(5)$ (E through C: $4 < 7$, replaced)	S, B, A, C
$E\,(4)$	$D\,(5), G\,(5)$	S, B, A, C, E
$D\,(5)$	$G\,(5)$ (G through D: $11 > 5$, kept)	S, B, A, C, E, D
$G\,(5)$	goal: return $S \to C \to G$, cost 5	

Table 3: UCS on fig. 1.

Informed search. **Informed search** adds a **distance approximation** $h(n)$ to the search: a function estimating the distance from a state n to a goal. It depends on the problem: for shortest path problems, e.g., the Manhattan or the Euclidean distance; for puzzle or ordering problems, the number of misplaced items. For the route from Munich to Berlin, h can be the straight-line distance, 504 km, while the least-cost road distance is 584 km. The approximation h itself is also called a **heuristic**.

Greedy search. **Greedy search** expands the frontier node with the least approximated distance $h(n)$ to a goal, i.e. the one assumed to be closest to a goal. It always makes a locally optimal choice in order to approximate a globally optimal solution, but it is a heuristic (definition 1.1): it is not guaranteed to find an optimal solution, it can lead straight to the wrong goal, and in the worst case it behaves like a

badly guided DFS. On the tree of fig. 2 it expands S, then 2 ($h = 13 < 25$), 6 ($h = 5$), 5 ($h = 6$), and finally pops G ($h = 0$) (table 4).

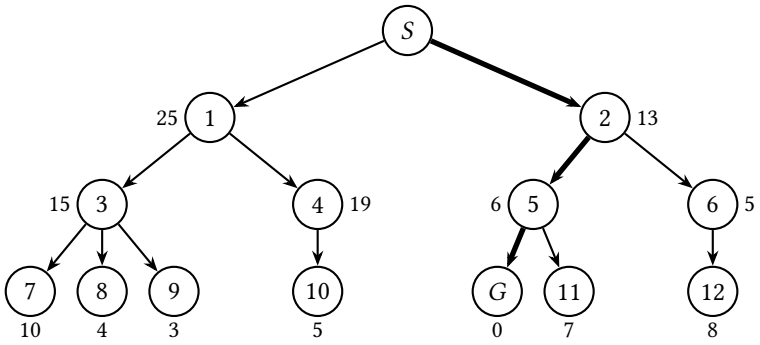


Figure 2: Greedy search example.

Expanded	Frontier after the step, $n\,(h)$
S	2 (13), 1 (25)
2	6 (5), 5 (6), 1 (25)
6	5 (6), 12 (8), 1 (25)
5	G (0), 11 (7), 12 (8), 1 (25)
G	goal: return $S \to 2 \to 5 \to G$

Table 4: Greedy search on fig. 2.

A* search. A* combines UCS and greedy search. UCS orders the frontier by the path cost $g(n)$ from the start to n (the actual "backward" cost), greedy search by the estimated "forward" cost $h(n)$ from n to a goal; **A* search** expands the frontier node with the least

$$f(n) = g(n) + h(n),$$

the estimated cost of a path from the start through n to a goal. As in UCS, the goal test is applied when a node is popped. On fig. 3 (table 5) A* returns $S \to A \to B \to G$ with cost 5, the least cost. Ordering by g alone, UCS would also expand E and F, cheap to reach but with no path to the goal, before popping G; ordering by h alone, greedy search would go from A to C, which has the smallest estimate $h(C) = 1$ but costs 6 to reach.

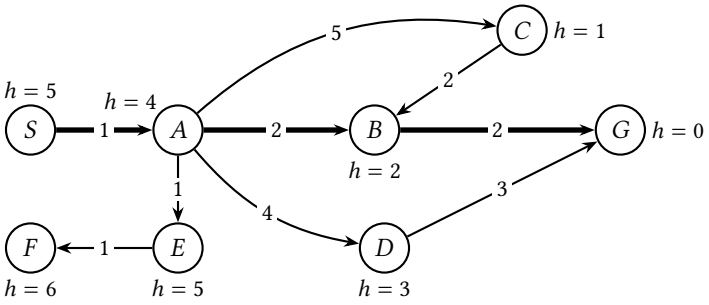


Figure 3: A* search example.

Expanded	Frontier after the step, $n : \ g + h = f$
S	$A : \ 1 + 4 = 5$
A	$B : \ 3 + 2 = 5, \quad C : \ 6 + 1 = 7, \quad E : \ 2 + 5 = 7, \quad D : \ 5 + 3 = 8$
B	$G : \ 5 + 0 = 5, \quad C : \ 7, \quad E : \ 7, \quad D : \ 8$
G	goal: return $S \to A \to B \to G$, cost 5

Table 5: A* search on fig. 3.

Admissibility. A* need not be optimal.

Example 1.7. Let $S \to A$ cost 1, $A \to G$ cost 3 and $S \to G$ cost 5, with $h(S) = 7$, $h(A) = 6$, $h(G) = 0$. After expanding S the frontier holds G with $f(G) = 5 + 0 = 5$ and A with $f(A) = 1 + 6 = 7$, so A* pops G and returns $S \to G$ with cost 5; it never expands A, although $S \to A \to G$ costs 4. The problem is that the estimate $h(A) = 6$ exceeds the true remaining cost 3 on the optimal path, so that $f(A)$ exceeds the actual cost 5 of the non-optimal path.

Definition 1.8 (Admissible heuristic)**.** Let $h^*(n)$ be the true cost from n to a nearest goal. A distance approximation h is **admissible** (**optimistic**) if

$$0 \le h(n) \le h^*(n) \qquad \text{for all nodes } n,$$

and **inadmissible** (**pessimistic**) otherwise.

The straight-line distance is admissible, since no road is shorter than the straight line ($504 \le 584$ above). In example 1.7, $h(A) = 6 > h^*(A) = 3$; the heuristic of fig. 3 is admissible ($h^*(C) = 4 \ge 1$, and $h = h^*$ at S, A, B, D, G). In practice, finding an admissible distance measure is often challenging.

Theorem 1.9 (Optimality of A* tree search)**.** *If h is admissible, A* organised as tree search is optimal.*

Proof. Let p^* be an optimal solution path, with cost c^*, ending in the goal G_1, and let G_2 be a suboptimal goal on the frontier, reached by a path of cost $g(G_2) > c^*$. We show that every node N of p^* (an ancestor of G_1) is expanded before G_2, so that the suboptimal path cannot become the solution. While G_1 has not been expanded, some node N of p^* is on the frontier, since the start node is on p^* and expanding a node of p^* puts its successor on p^* on the frontier.

1. $f(N) \le f(G_1)$: by admissibility $h(N) \le h^*(N)$, which is at most the cost from N to G_1 along p^*, so $f(N) = g(N) + h(N) \le c^* = g(G_1)$; and $g(G_1) = f(G_1)$ because $h = 0$ at a goal.
2. $f(G_1) < f(G_2)$: G_2 is suboptimal, so $g(G_1) < g(G_2)$, and again $h = 0$ at both goals.
3. Hence $f(N) \le f(G_1) < f(G_2)$, and N is expanded before G_2.

Applying this to every node of p^*, all ancestors of G_1 are expanded before G_2, and then G_1 itself is popped before G_2 since $f(G_1) < f(G_2)$. So A* returns the optimal path. □

UCS expands equally in all "directions", in contours of increasing g around the start; A* expands mainly towards the goal, but hedges its bets enough to ensure optimality.

Algorithm	Expands first	Frontier	Guarantee
DFS	deepest node	LIFO stack	not optimal (table 1)
BFS	shallowest node	FIFO queue	shortest path (fewest actions)
UCS	least $g(n)$	priority queue	least-cost path
Greedy	least $h(n)$	priority queue	none (heuristic)
A*	least $f(n) = g(n) + h(n)$	priority queue	least-cost path if h admissible (tree search)

Table 6: Search algorithms compared.

1.3 Adversarial and expectimax search

Agents and utilities. An **agent** is any system that perceives its environment and takes actions within it; it uses prior knowledge, past experience, goals and observations, and receives feedback (outcomes) from the environment. An agent always maximises its utility.

Definition 1.10 (Utility, lottery)**.** A **utility function** U maps outcomes (states of the environment) $s \in S$ to real numbers describing the agent's satisfaction with them. A situation is **deterministic** if it has only one possible, predefined outcome, and a **lottery** if one of several outcomes is drawn at random; a lottery with outcomes x_i and probabilities p_i is written

$$L = [p_1, x_1;\ p_2, x_2;\ \ldots;\ p_n, x_n].$$

A utility satisfies

$$U(s_1) \geq U(s_2) \iff s_1 \succeq s_2, \qquad U([p_1, s_1; \ldots; p_n, s_n]) = \sum_i p_i\, U(s_i).$$

The agent prefers ($\succeq$) a state of higher utility, and the utility of a lottery is its **expected utility**, the sum of the utilities weighted by their probabilities. A **rational agent** chooses, based on its knowledge, the action that maximises its expected utility.

For instance, flipping a coin is the lottery $[0.5, \text{head};\ 0.5, \text{tail}]$. Utilities represent the agent's reward (e.g. $+1$ for winning a game and -1 for losing), and any "rational" preferences can be summarised by a utility function. A **positive affine transformation** of a utility is again a utility, representing the same preferences:

$$U'(s) = k_1 U(s) + k_2, \qquad k_1 > 0, \tag{1}$$

since U' orders states like U, and $\sum_i p_i U'(s_i) = k_1 \sum_i p_i U(s_i) + k_2$ orders lotteries like U.

Adversarial games. In **zero-sum games** the agents have opposite utilities (values on outcomes): agent A wants to maximise the objective value and agent B to minimise it, so they compete against each other. In **non-zero-sum games** the agents have independent utilities, and other scenarios are possible, e.g. cooperation (both maximise the same objective) or indifference (the objectives are unrelated). Here two players, **MAX** and **MIN**, play a zero-sum game, alternating turns (tic-tac-toe, chess, checkers).

Definition 1.11 (Game tree)**.** A two-player game is modelled by a search tree whose nodes are the states of the game, $S = S^{\text{MAX}} \cup S^{\text{MIN}} \cup S^T$ (states where MAX or MIN moves, and terminal states), with the start of the game as root. Arcs are the actions of the player in turn:

$$a \in A^{\text{MAX}},\ s \xrightarrow{a} s' \ \Rightarrow\ s \in S^{\text{MAX}},\ s' \in S^{\text{MIN}} \cup S^T; \qquad a \in A^{\text{MIN}},\ s \xrightarrow{a} s' \ \Rightarrow\ s \in S^{\text{MIN}},\ s' \in S^{\text{MAX}} \cup S^T.$$

A node is a goal (terminal) node when the game is over, e.g. in tic-tac-toe when a player wins or no free cell is left (draw), and a **utility function** gives the value of each terminal state (-1, 0, $+1$ for loss, draw, win of MAX). A solution is a **strategy**: the best action in each state.

Definition 1.12 (Minimax value)**.** The **value** $V(s)$ of a state is the best achievable outcome (utility) from s. The **minimax value** is the best utility the player in turn can achieve against a rational (optimal) adversary who moves in every second layer:

$$V(s) = \begin{cases} U(s), & s \in S^T \text{ (known)}, \\ \max_{s' \in \text{successors}(s)} V(s'), & s \in S^{\text{MAX}} \text{ (agent's control)}, \\ \min_{s' \in \text{successors}(s)} V(s'), & s \in S^{\text{MIN}} \text{ (opponent's control)}. \end{cases}$$

Minimax search generates the complete game tree (all playing sequences) with DFS, computes the utility of each terminal state, propagates the values up the tree (minimum of the successors at MIN nodes, maximum at MAX nodes), and from the initial state MAX chooses the move to the successor with the highest value. In fig. 4(a) the MIN nodes get $\min(8,2) = 2$ and $\min(5,6) = 5$, and the root $\max(2,5) = 5$: MAX moves right.

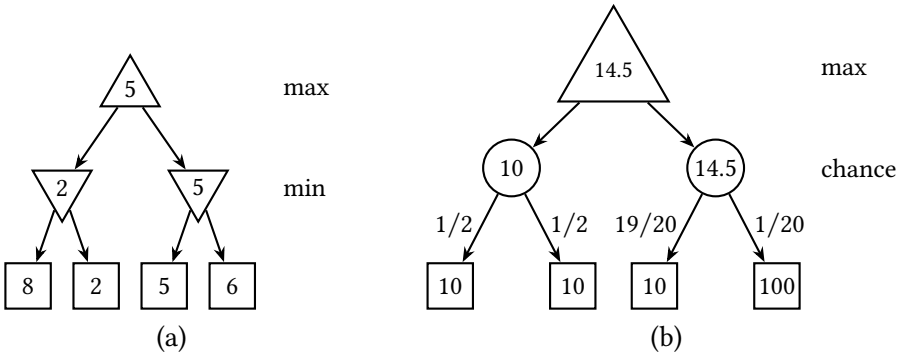


Figure 4: (a) Minimax tree; (b) expectimax tree.

Since minimax is DFS, it has time complexity $O(b^m)$ and space complexity $O(b\,m)$, with b the maximal number of successors and m the depth (number of layers) of the tree. For chess $b \approx 35$ and $m \approx 100$, so the exact solution is far too expensive: a chess program needs about 100 seconds to explore 8 layers.

Depth-limited search. When the tree is too big to explore completely, **depth-limited search** runs DFS only to a limited depth and applies an **evaluation function** to the non-terminal positions there, which act as new terminal states. This loses the guarantee of optimal play; the more layers are considered, the closer the strategy may get to the optimal one. For instance, if the evaluation function gives $-1, -2$ below the left MIN node and $4, 9$ below the right one, the MIN nodes get -2 and 4, and the root 4.

Expectimax search. Outcomes can be uncertain because of explicit randomness (rolling dice), unpredictable opponents (the ghosts in Pac-Man respond randomly) or actions that fail (the wheels of a robot slip). **Expectimax search** then computes the average score under optimal play instead of the worst case: besides MAX nodes the tree has **chance nodes**, whose value is the expected utility of their children, the probability-weighted average; a rational agent picks the action of maximal expected utility. In fig. 4(b) the two actions lead to the lotteries $[\frac{1}{2}, 10;\ \frac{1}{2}, 10]$ and $[\frac{19}{20}, 10;\ \frac{1}{20}, 100]$, with values

$$\tfrac{1}{2}\cdot 10 + \tfrac{1}{2}\cdot 10 = 10, \qquad \tfrac{19}{20}\cdot 10 + \tfrac{1}{20}\cdot 100 = 9.5 + 5 = 14.5,$$

so MAX takes the right action and the root value is 14.5. This uncertainty will be formalised by Markov decision processes in the next deck.

Transformations of utilities. For minimax only the ordering of the terminal values matters, not their scale: more valuable states must have higher utilities, and any increasing transformation φ leaves the decisions unchanged, since $\varphi(\max(x,y)) = \max(\varphi(x), \varphi(y))$ and likewise for min (**insensitivity to monotonic transformations**). For expectimax the scale and magnitudes matter, because they change the ordering of the weighted sums.

Example 1.13. MAX chooses between the lotteries $[0.4, 0;\ 0.6, 40]$ and $[0.5, 20;\ 0.5, 30]$. Their expected utilities are $0.4 \cdot 0 + 0.6 \cdot 40 = 24 < 25 = 0.5 \cdot 20 + 0.5 \cdot 30$, so expectimax takes the second. After the transformation $x \mapsto x^2$, increasing on these nonnegative utilities, the lotteries become $[0.4, 0;\ 0.6, 1600]$ and $[0.5, 400;\ 0.5, 900]$, with $0.6 \cdot 1600 = 960 > 650 = 0.5 \cdot 400 + 0.5 \cdot 900$, and expectimax takes the first. Minimax would value both lotteries by their worst outcome and choose the same action before and after

the transformation. Only positive affine transformations (1) are guaranteed to preserve the expectimax decision.

2 Markov Decision Processes

The search problems of section 1 are deterministic: an action leads to a known successor. A Markov decision process models non-deterministic problems, in which an action leads to one of several states at random; its solution is no longer a plan but a policy, which fixes an action for every state.

2.1 MDP model and policies

In a **Markov decision process** (**MDP**) an agent takes actions in an environment, which responds to each action with a new state and a reward: at time t the agent is in state s_t and takes action a_t, and the environment returns the next state s_{t+1} and the reward r_t.

Definition 2.1 (Markov decision process)**.** An MDP consists of a **state space** S; for each state an **action space** $A(s)$; a **transition function**

$$T(s, a, s') = P(s' \mid s, a), \qquad s, s' \in S,\ a \in A(s),$$

the probability that action a taken in s leads to s'; a **reward function** $R(s, a, s')$, the reward for this transition; a **start state** $s_0 \in S$, where the process begins; possibly **terminal states**, where it ends (a terminal state can be infinitely far in the future, and an MDP need not have one); and a discount factor γ (definition 2.6). The reward of step t is $r_t = R(s_t, a_t, s_{t+1})$.

In section 1, $T(s, a)$ was the unique successor of s under a; now $T(s, a, \cdot)$ is a probability distribution over successors.

Example 2.2 (Recycling robot)**.** A robot searches for cans in a park (fig. 5). Its states are the battery levels $S = \{\text{high}, \text{low}\}$, its actions $A(\text{high}) = \{\text{search}, \text{wait}\}$ and $A(\text{low}) = \{\text{search}, \text{wait}, \text{recharge}\}$. Searching earns reward 4, waiting 1, recharging 0. With high battery, searching keeps the level high with probability 0.1 and lowers it with probability 0.9; with low battery, searching keeps it low with probability 0.8, and with probability 0.2 the robot runs out of energy: it gets reward -7, is recharged and starts again with high battery. Waiting leaves the level unchanged, recharging brings it from low to high. For instance $T(\text{low}, \text{search}, \text{high}) = 0.2$ and $R(\text{low}, \text{search}, \text{high}) = -7$. The robot starts with high battery, and there is no terminal state. The goal is to collect as many cans as possible, i.e. to find the best action in each state.

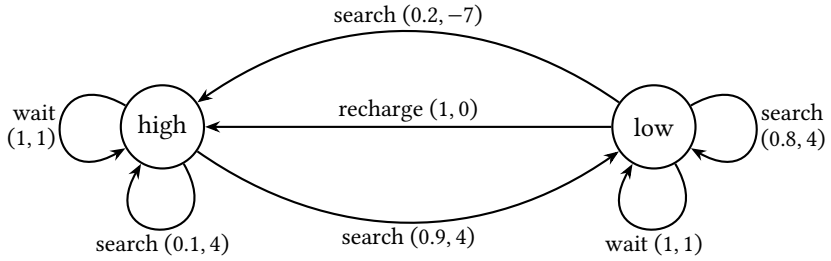


Figure 5: Recycling robot.

Markov chains, reward processes and decision processes. A **Markov chain** (**Markov process**) is a memoryless random process, a sequence of random states; a **Markov reward process** (**MRP**) is a Markov chain with rewards; an MDP adds the agent's choice of actions (table 7). For instance, a chain may move from s_1 to s_2, s_3, s_4 with probabilities 0.2, 0.5, 0.3; an MRP attaches rewards 3, 5, 2 to these transitions; in an MDP the same distribution and rewards are the outcomes of one action a chosen in s_1.

Model	Actions	Transition function	Reward function	Discount
Markov chain	none	$T(s, s') = P(s' \mid s)$	none	none
MRP	none	$T(s, s') = P(s' \mid s)$	$R(s, s')$	$\gamma \in [0, 1]$
MDP	$A(s)$	$T(s, a, s') = P(s' \mid s, a)$	$R(s, a, s')$	$\gamma \in [0, 1]$

Table 7: Markov chain, Markov reward process and Markov decision process.

Definition 2.3 (Markov property)**.** A random process with states S_t has the **Markov property** if, given the present state, the future is independent of the past, i.e. future states depend only on the present state:

$$P(S_{t+1} = s' \mid S_t = s_t, S_{t-1} = s_{t-1}, \dots, S_0 = s_0) = P(S_{t+1} = s' \mid S_t = s_t).$$

A model with the Markov property can be described as a Markov chain (**Markov assumption**). In an MDP the effect of an action depends only on the state in which it is taken, not on the history of earlier states and actions:

$$P(S_{t+1} = s' \mid S_t = s_t, A_t = a_t, S_{t-1} = s_{t-1}, A_{t-1} = a_{t-1}, \dots, S_0 = s_0) = P(S_{t+1} = s' \mid S_t = s_t, A_t = a_t).$$

For example, a chain with two states E and A that moves from E to E or A with probabilities 0.3 and 0.7, and from A to E or A with probabilities 0.6 and 0.4, has the Markov property: the next state depends only on the current one.

Reward allocation. The reward can be attached to being in a state, $R(s)$, to taking an action in a state, $R(s, a)$, or to taking an action in a state and arriving in a given successor, $R(s, a, s')$. All three are correct but model different reward allocation processes (when the agent receives the reward); $R(s, a, s')$ is the most general and is used here.

Episodic and continuous tasks. An **episodic task** has a natural notion of a final time step: the interaction with the environment breaks naturally into subsequences, the **episodes**, which end either in a terminal state (**indefinite horizon**) or after a predefined number of time steps (**finite horizon**). A **continuous** (**continuing**) **task** has neither terminal states nor a fixed number of steps and can go on forever (**infinite horizon**); the recycling robot is an example.

Example 2.4 (Gridworld)**.** The **gridworld** is a grid of 4×3 cells (x, y), $x = 1, \dots, 4$, $y = 1, \dots, 3$, each cell a state (figs. 6 and 8). A robot starts in the cell $(1, 1)$ and wants to maximise its reward. Its actions are left, right, up, down; with probability 0.8 it moves in the intended direction, and with probability 0.1 each it slips at a right angle to the left or to the right of it. A move into the wall cell $(2, 2)$ or out of the grid leaves the robot where it is. The cells $(4, 3)$, with reward $+1$, and $(4, 2)$, with reward -1, are terminal; all other states have reward 0. For instance, the action up in $(1, 1)$ leads to $(1, 2)$ with probability 0.8, to $(2, 1)$ with probability 0.1, and keeps the robot in $(1, 1)$ with probability 0.1 (slip to the left, out of the grid). The gridworld is an episodic task with terminal states.

Policies. In deterministic single-agent search the objective is an optimal plan, a sequence of actions from the start to a goal; expectimax computes only the best action in a single state. In an MDP the objective is an optimal policy, which specifies the action for every state. (In section 1, π denoted a plan; from now on it denotes a policy.)

Definition 2.5 (Policy, optimisation objective)**.** A **policy** $\pi : S \to A$ maps every state to the action the agent chooses in it. A policy can also be **stochastic**: a conditional probability distribution $\pi(a \mid s)$ over the actions in each state, with $\sum_a \pi(a \mid s) = 1$. Given the start state s_0, the agent maximises its expected total reward over all time steps; with Π the set of all policies and horizon H,

$$\pi^* = \arg\max_{\pi \in \Pi} \mathbb{E}_\pi \Big[\sum_{t=0}^{H-1} R(s_t, a_t, s_{t+1}) \,\Big|\, s_0 \Big],$$

where the expectation is over the states and actions generated by following π from s_0. Such a π^* is an **optimal policy**: it maximises the expected utility.

For the recycling robot, high $\mapsto$ search, low $\mapsto$ recharge is a policy. In the gridworld, a policy is an arrow in each cell, e.g. right in the top row, up in $(1,1)$, $(1,2)$ and $(3,2)$, left in $(2,1)$, $(3,1)$ and $(4,1)$. A stochastic policy is useful, for example, when the underlying MDP is unknown, or in a game like rock, paper, scissors.

2.2 Utilities and the Bellman equations

Discounting. The utility of being in a state depends on the rewards that the agent can reach from it in the future. It is reasonable to maximise the sum of rewards, and also to prefer rewards now to rewards later (after fewer actions): the agent may not know how many time steps are left, fast feedback helps to learn a policy, the further the agent plans into the future the more uncertainty it must consider, and in a game that lasts forever the sum of rewards would be infinite. Discounting makes the values of rewards decay exponentially: with a **discount factor** $0 \le \gamma \le 1$, a reward n steps away is multiplied by γ^n, once for each step into the future.

Definition 2.6 (Discounted utility)**.** The **utility** of a sequence of realised rewards is the sum of the discounted rewards,

$$U([r_0, r_1, r_2, \dots]) = \sum_{t=0}^{\infty} \gamma^t r_t.$$

In definition 1.10 a utility was attached to an outcome; here it is attached to the sequence of rewards collected from a state on. (From the policy-gradient deck on, the discounted sum of realised rewards is written G.)

Example 2.7. From a state A two routes lead back to A with reward 6. The short route goes from A to H, then to I with probability 0.9^5, and from I back to A with reward 6: the reward comes with the third transition. The long route goes through B, C, D, E, F, G, each of $B, \dots, F$ continuing with probability 0.9, and from G back to A with reward 6: the reward comes with the seventh transition. Both routes succeed with probability 0.9^5; otherwise the agent falls into a state J, from which it returns to A with reward -1, and all other rewards are 0. The agent prefers the short route, because its reward comes earlier. With $\gamma = 0.8$ and start in A, the reward 6 of the long route counts $\gamma^6 \cdot 6 = 0.8^6 \cdot 6 \approx 1.57$, while that of the short route counts $\gamma^2 \cdot 6 = 3.84$.

Infinite problems. If the process may last forever, there are three ways to avoid infinite reward:

- **Discounting** with $0 < \gamma < 1$. If every reward is at most $R_{\max}$, then

$$U([r_0, r_1, \dots]) = \sum_{t=0}^{\infty} \gamma^t r_t \le \sum_{t=0}^{\infty} \gamma^t R_{\max} = \frac{R_{\max}}{1-\gamma}, \tag{2}$$

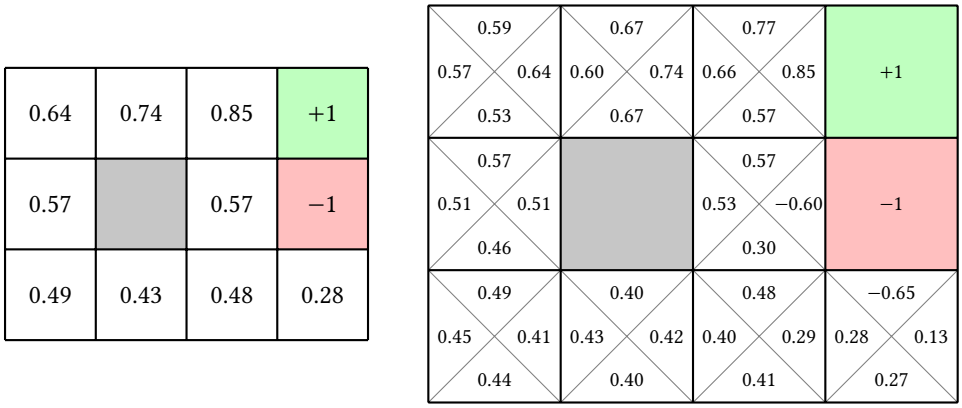
  by the geometric series: $S_n = \sum_{t=0}^{n} \gamma^t$ satisfies $S_n - \gamma S_n = 1 - \gamma^{n+1}$, so $S_n = \frac{1-\gamma^{n+1}}{1-\gamma} \to \frac{1}{1-\gamma}$ as $n \to \infty$, since $\gamma^{n+1} \to 0$. A smaller γ means a shorter "horizon".
- **Finite horizon**: episodes terminate after H steps (e.g. a lifetime), similarly to depth-limited search. This gives **nonstationary** policies, since the best action depends on the time left.
- **Absorbing state**: it is guaranteed that, under every policy, a terminal state is eventually reached.

Definition 2.8 (V-values and Q-values)**.** For a policy π, the **V-value** $V^\pi(s)$ is the expected utility (expected discounted return) when starting in state s and following π, and the **Q-value** $Q^\pi(s,a)$ the expected utility when starting in s, taking action a, and following π afterwards. The **optimal values**

$$V^*(s) = \mathbb{E}_{\pi^*,\, s' \sim P}[U], \qquad Q^*(s,a) = \mathbb{E}_{\pi^*,\, s' \sim P}[U \mid a]$$

are the expected utilities of being in s, respectively of taking a in s, and acting optimally thereafter; the expectation is over the actions of π^* and the random successors $s' \sim P(\cdot \mid s, a)$.

This section mostly uses V-values; Q-values return with reinforcement learning. Figure 6 shows the optimal values of the gridworld with noise 0.2 (the probability of slipping), $\gamma = 0.9$ and **living reward** 0 (the reward of every non-terminal step). The reward of a terminal cell is collected when the episode ends in it, so $V(4,3) = +1$ and $V(4,2) = -1$, and all other transitions have reward 0.



(a) (b)

Figure 6: Optimal values of the gridworld: (a) V^*; (b) Q^*, one triangle per action.

Utilities in a search tree. The utilities can be computed in a search tree with expectimax (fig. 7): the states are max nodes (**V-states**), and choosing an action a in s leads to a chance node, the **Q-state** (s, a), whose children are the successors s' with probabilities $T(s, a, s')$. This has two problems. States are repeated (in the robot's tree, high and low appear on every level), so needed quantities should be computed only once; and the tree goes on forever, so one would compute to a limited depth, increasing it until the discounted reward changes little; deep parts of the tree eventually do not matter if $\gamma < 1$. Building this tree is very costly; the Bellman equations avoid it.

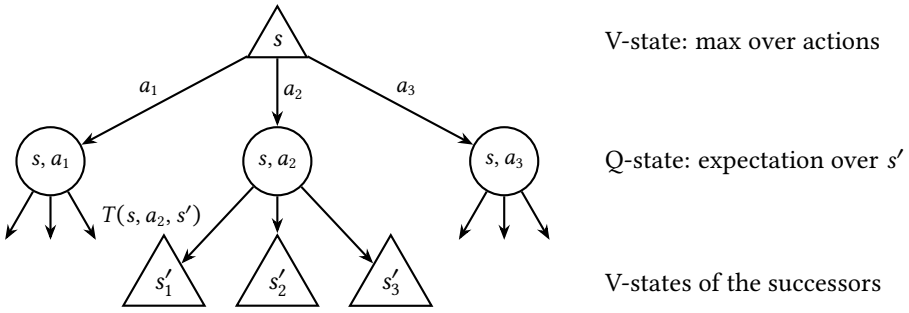


Figure 7: One-step look-ahead in an MDP.

Bellman equations. The **Bellman equations** give the utility of every state and state-action pair under a policy, following the idea "take the correct first action and keep being optimal afterwards". The utility from time t on, $U_t = \sum_{k \geq 0} \gamma^k r_{t+k}$, splits into the first reward and the discounted utility from $t+1$ on:

$$U_t = r_t + \gamma\,(r_{t+1} + \gamma r_{t+2} + \cdots) = r_t + \gamma\, U_{t+1}.$$

Taking expectations along the optimal path gives a one-step look-ahead relationship among the optimal utility values, i.e. a recursive expectimax: the Q-state averages over the successors, and the V-state takes the best action,

$$Q^*(s,a) = \sum_{s'} T(s,a,s') \left[R(s,a,s') + \gamma V^*(s')\right], \tag{3}$$

$$V^*(s) = \max_a Q^*(s,a) = \max_a \sum_{s'} T(s,a,s') \left[R(s,a,s') + \gamma V^*(s')\right]. \tag{4}$$

For instance, in the gridworld (fig. 6) the action right in $(3,3)$ reaches $(4,3)$ with probability 0.8, stays in $(3,3)$ (slip up, out of the grid) with probability 0.1, and reaches $(3,2)$ with probability 0.1:

$$Q^*((3,3),\text{right}) = 0.8 \cdot 0.9 \cdot 1 + 0.1 \cdot 0.9 \cdot 0.85 + 0.1 \cdot 0.9 \cdot 0.57 \approx 0.85 = V^*(3,3).$$

2.3 Value iteration and policy iteration

Motivation. With $|S|$ states and $|A|$ actions per state there are $|A|^{|S|}$ deterministic policies (in the gridworld, $4^9 = 262144$ on the 9 non-terminal cells), so trying all of them is hopeless. Two ideas lead to iterative algorithms, and both compute the optimal utilities, from which the optimal policy follows.

- **Value iteration**: iterate the **Bellman update** $V_{i+1} = B(V_i)$, the right-hand side of (4) applied to V_i. It is a contraction, $\|B(V) - B(V')\| \le \zeta \|V - V'\|$ with a constant $\zeta < 1$, so by the **Banach fixed-point theorem** the iterates converge to its unique fixed point, V^*.
- **Policy iteration**: there are only finitely many policies; if every iteration improves the policy, the optimal policy is reached.

Value iteration. Starting from an arbitrary value function, value iteration repeats the Bellman update in every state until the values converge, and then extracts the policy.

Algorithm 4 Value iteration

Require: MDP (S, A, T, R, γ), threshold $\Theta > 0$
initialise $V_0(s)$ arbitrarily for all $s \in S$; $i \leftarrow 0$
repeat
  for each $s \in S$ **do**
    $V_{i+1}(s) \leftarrow \max_a \sum_{s'} T(s,a,s') \left[R(s,a,s') + \gamma V_i(s')\right]$ ▷ Bellman update
  end for
  $i \leftarrow i + 1$
until the change between V_{i-1} and V_i is smaller than Θ
for each $s \in S$ **do** ▷ policy extraction
  $\pi(s) \leftarrow \arg\max_a \sum_{s'} T(s,a,s') \left[R(s,a,s') + \gamma V_i(s')\right]$
end for

Each update combines policy improvement, since it chooses the best action, with policy evaluation, since it assigns new utilities. The Bellman equation (4) characterises the optimal values; value iteration uses it as an update to compute them, as a fixed-point method. **Policy extraction** retrieves the policy from the values by a one-step expectimax,

$$\pi^*(s) = \arg\max_{a \in A(s)} \sum_{s'} T(s,a,s') \left[R(s,a,s') + \gamma V^*(s')\right],$$

which is computationally expensive, since every action needs a sum over the successors. The infinite-horizon optimal policy is **stationary**: the optimal action in a state is the same at all times.

Example 2.9 (Value iteration on the gridworld)**.** Start with $V_0 = 0$ on the gridworld of example 2.4, with $\gamma = 0.9$. In iteration 1 only the terminal cells change, $V_1(4,3) = +1$ and $V_1(4,2) = -1$; every other update is 0. In iteration 2 the only new value is in $(3,3)$, where the action right gives

$$V_2(3,3) = 0.8 \cdot 0.9 \cdot V_1(4,3) + 0.1 \cdot 0.9 \cdot V_1(3,3) + 0.1 \cdot 0.9 \cdot V_1(3,2) = 0.72.$$

In iteration 3 the values spread one more cell:

$$V_3(3,3) = 0.8 \cdot 0.9 \cdot 1 + 0.1 \cdot 0.9 \cdot 0.72 + 0.1 \cdot 0.9 \cdot 0 = 0.7848 \approx 0.78 \quad \text{(right)},$$
$$V_3(2,3) = 0.8 \cdot 0.9 \cdot 0.72 + 0.1 \cdot 0.9 \cdot 0 + 0.1 \cdot 0.9 \cdot 0 = 0.5184 \approx 0.52 \quad \text{(right)},$$
$$V_3(3,2) = 0.8 \cdot 0.9 \cdot 0.72 + 0.1 \cdot 0.9 \cdot 0 + 0.1 \cdot 0.9 \cdot (-1) = 0.4284 \approx 0.43 \quad \text{(up)},$$

where in $(3, 2)$ the slip to the left hits the wall and keeps the robot in place, and the slip to the right ends in -1. Figure 8 shows V_1, V_2, V_3 with the greedy policy for each (ties, all actions worth 0, broken in the order up, left, down, right); after about 100 iterations the values have converged to V^* of fig. 6(a), and policy extraction gives the optimal policy of fig. 8(d).

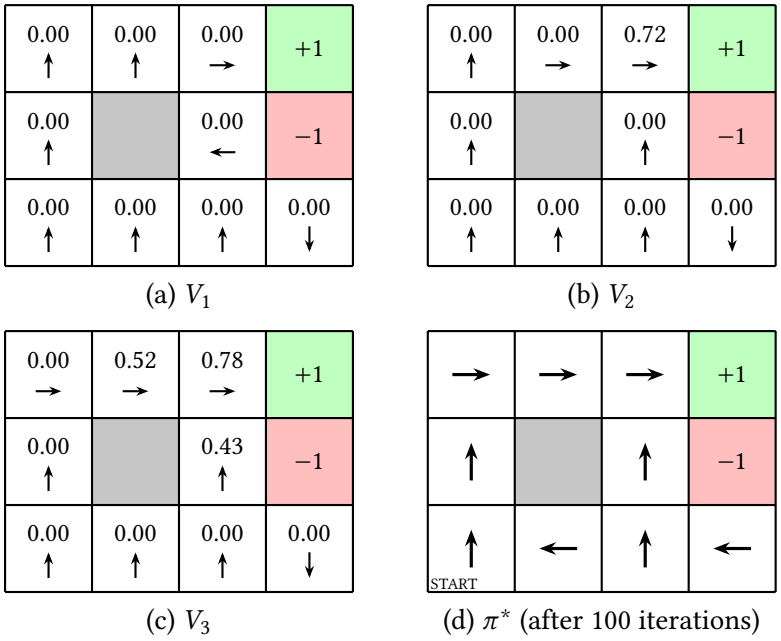


Figure 8: Value iteration on the gridworld.

Problems of value iteration. Every iteration computes the maximum over all state-to-state transition distributions, which costs $O(|S|^2|A|)$. Moreover, the policy often converges long before the values: the values change only slightly, and slight changes do not alter the argmax (one-step look-ahead values 4.235, 1.678, 2.495 that become 4.287, 1.738, 2.123 still select the first action).

Policy iteration. Policy iteration starts from an arbitrary policy and alternates **policy evaluation**, which computes the utilities V^{π_k} of the current policy, with **policy improvement**, which chooses the best action in every state by a one-step look-ahead on these fixed utilities; it stops when the policy no longer changes. The utilities $V^\pi(s)$ of a given policy (definition 2.8) can be computed by iterating the Bellman equations of π from $V_0^\pi(s) = 0$, as in the inner loop of algorithm 5, at a cost of $O(|S|^2)$ per iteration. Alternatively, since the policy is fixed there is no maximisation, and the Bellman equations reduce to a **linear system** of $|S|$ equations in the $|S|$ unknowns $V^\pi(s)$:

$$V^\pi(s) = \sum_{s'} T(s, \pi(s), s') \left[R(s, \pi(s), s') + \gamma V^\pi(s')\right], \qquad s \in S. \tag{5}$$

Small state and action spaces are solved with the linear system, large ones iteratively.

Algorithm 5 Policy iteration

Require: MDP (S, A, T, R, γ), threshold $\Theta > 0$

initialise $V(s)$ and $\pi_0(s)$ arbitrarily for all $s \in S$; $k \leftarrow 0$
loop
  repeat ▷ policy evaluation of π_k
    for each $s \in S$ **do**
      $V_{i+1}^{\pi_k}(s) \leftarrow \sum_{s'} T(s, \pi_k(s), s') \left[R(s, \pi_k(s), s') + \gamma V_i^{\pi_k}(s')\right]$
    end for
  until the change between $V_i^{\pi_k}$ and $V_{i+1}^{\pi_k}$ is smaller than Θ
  for each $s \in S$ **do** ▷ policy improvement
    $\pi_{k+1}(s) \leftarrow \arg\max_a \sum_{s'} T(s, a, s') \left[R(s, a, s') + \gamma V^{\pi_k}(s')\right]$ ▷ in a tie, keep $\pi_k(s)$
  end for
  if $\pi_{k+1} = \pi_k$ **then terminate** ▷ π_k is optimal
  end if
  $k \leftarrow k + 1$
end loop

Theorem 2.10 (Convergence of policy iteration)**.** *Policy iteration is guaranteed to converge, and at convergence the current policy is optimal and its V-values are the optimal ones.*

Proof sketch. Convergence. The policy improves in every iteration in which it changes (in a tie the current action is kept), so no policy is encountered twice. Since there are at most $|A|^{|S|}$ policies, after finitely many iterations $\pi_{k+1}(s) = \pi_k(s)$ for all states s. *Optimality.* At convergence $\pi_k(s)$ attains the maximum in the improvement step, so the evaluation equation (5) of π_k becomes

$$V^{\pi_k}(s) = \max_a \sum_{s'} T(s, a, s') \left[R(s, a, s') + \gamma V^{\pi_k}(s')\right] \qquad \text{for all } s,$$

so V^{π_k} satisfies the Bellman equation (4), whose only solution is the fixed point V^*. Hence $V^{\pi_k} = V^*$ and π_k is optimal. □

Example 2.11 (Policy iteration on the recycling robot)**.** Take the robot of example 2.2 with $\gamma = 0.9$ and start from $\pi_0 = (\text{high} \mapsto \text{search},\ \text{low} \mapsto \text{recharge})$; write V_h, V_l for the values of high and low. Evaluating π_0 with (5):

$$V_h = 0.1\,(4 + 0.9 V_h) + 0.9\,(4 + 0.9 V_l), \qquad V_l = 0 + 0.9 V_h,$$

so $V_h = 4 + 0.09 V_h + 0.729 V_h$, i.e. $0.181\ V_h = 4$: $V_h \approx 22.10$ and $V_l \approx 19.89$. The improvement step (first row of table 8) keeps search in high and changes the action in low, where searching has the one-step look-ahead value

$$0.8\,(4 + 0.9 \cdot 19.89) + 0.2\,(-7 + 0.9 \cdot 22.10) = 20.10,$$

more than recharging ($0.9 \cdot 22.10 = 19.89$) and waiting ($1 + 0.9 \cdot 19.89 = 18.90$). So $\pi_1 = (\text{search},\ \text{search})$: with low battery, searching is worth the risk of running out. Evaluating π_1,

$$V_h = 0.1\,(4 + 0.9 V_h) + 0.9\,(4 + 0.9 V_l), \qquad V_l = 0.8\,(4 + 0.9 V_l) + 0.2\,(-7 + 0.9 V_h),$$

i.e. $0.91\ V_h - 0.81\ V_l = 4$ and $-0.18\ V_h + 0.28\ V_l = 1.8$, gives $V_h \approx 23.65$, $V_l \approx 21.63$. The next improvement step keeps search in both states (table 8), so $\pi^* = \pi_1$ and $V^* = (23.65,\ 21.63)$.

	V^{π_k}		high		low		
π_k (high, low)	V_h	V_l	search	wait	search	wait	recharge
π_0 (search, recharge)	22.10	19.89	**22.10**	20.89	**20.10**	18.90	19.89
π_1 (search, search)	23.65	21.63	**23.65**	22.29	**21.63**	20.47	21.29

Table 8: Policy iteration on the recycling robot ($\gamma = 0.9$): values of π_k and one-step look-ahead values, maximum in bold.

Comparison. Value iteration and policy iteration both compute the correct V-values and extract the optimal policy, both use one-step look-ahead expectimax fragments, i.e. variants of the Bellman update, and both can be implemented as a dynamic program to solve an MDP (table 9). Reinforcement learning, the subject of the next deck, learns an optimal policy without knowing the transition and reward functions.

	Value iteration	Policy iteration
Starts from	an arbitrary value function	an arbitrary policy
One iteration	Bellman update with max over actions: updates the values and, implicitly, the policy	several passes with the policy fixed (fast, one action per state, or a linear system), then one improvement step (as slow as a value-iteration pass)
Cost per pass	$O(\|S\|^2\|A\|)$	$O(\|S\|^2)$ per evaluation pass
Policy	not tracked; extracted once at the end	explicit, improved in every iteration
Stops when	the values change by less than Θ	the policy no longer changes

Table 9: Value iteration and policy iteration compared.